



Alan West

Posted on Mar 29



3

How to Recover from a Corrupted Git Repository

[#git](#) [#programming](#) [#devops](#) [#tutorial](#)

There's a special kind of dread that hits when you run `git status` and get back something like `fatal: bad object HEAD` or `error: object file is empty`. Your stomach drops. Your commit history — weeks of work — feels like it just vanished.

I've been there three times in eight years. Twice from disk failures, once from a VM that got killed mid-push. Every time, I thought I'd lost everything. Every time, I was wrong. Git is surprisingly resilient, and most "corrupted" repos are fully recoverable if you know where to look.

Why Git Repositories Get Corrupted

Before we fix anything, it helps to understand what actually broke. Git stores everything as objects in `.git/objects/` — blobs (file contents), trees (directories), commits, and tags. Each object is named by its SHA-1 hash and compressed with zlib.

Corruption usually happens when:

- **Disk failure or power loss** interrupts a write to `.git/objects/`
- **A process gets killed** during `git gc`, `git repack`, or a push/pull
- **Filesystem bugs** (more common on networked/shared drives than you'd think)
- **Aggressive antivirus or backup software** locks or modifies files in `.git/`

The good news: Git's content-addressable storage means that if an object exists and its hash checks out, the data is intact. Corruption is usually isolated to a handful of objects.

Step 1: Assess the Damage

First, figure out what's actually broken. Don't panic-delete anything.

```
# Check the overall integrity of the repository
git fsck --full --no-dangling
```

```
# You'll see output like:
# error: object file .git/objects/a1/b2c3... is empty
# error: sha1 mismatch for .git/objects/d4/e5f6...
# missing blob a1b2c3d4e5f6...
# broken link from tree f7a8b9... to blob a1b2c3...
```

This tells you exactly which objects are damaged and what type they are. Write down the broken SHA hashes — you'll need them.

```
# Also check if HEAD itself is broken
cat .git/HEAD
# Should show something like: ref: refs/heads/main

# Then check if that ref points to a valid commit
cat .git/refs/heads/main
git cat-file -t $(cat .git/refs/heads/main)
# Should output: commit
```

If `HEAD` or your branch ref is corrupted, that's your starting point. If it's deeper objects (blobs or trees), you have more options.

Step 2: Recover from the Reflog

Git's reflog is your best friend here. It keeps a local log of where your branch pointers have been, and it survives most corruption scenarios because it's stored separately from the object database.

```
# List recent reflog entries
git reflog show --all

# If HEAD is broken, try reading the reflog directly
cat .git/logs/HEAD

# Find the last known good commit hash and reset to it
git reset --hard <good-commit-hash>
```

The reflog typically keeps 90 days of history by default. If your corruption is recent (and it usually is), there's almost certainly a valid commit hash sitting in that log.

Step 3: Recover Objects from a Remote

If you've been pushing to any remote — GitHub, a self-hosted server, a colleague's machine — you already have a backup. This is the easiest recovery path.

```
# First, move the broken objects out of the way
mkdir -p .git/objects-broken

# For each broken object file identified by git fsck,
# move it to the backup directory
# Example: if a1b2c3... is broken
mv .git/objects/a1/b2c3d4e5f6* .git/objects-broken/

# Now fetch from your remote to re-download the missing objects
git fetch origin

# Run fsck again to verify
git fsck --full --no-dangling
```

This works because Git objects are immutable and content-addressed. The same commit on your remote has the exact same SHA hash and the exact same bytes. Fetching pulls down any objects you're missing.

Step 4: Recover Empty or Damaged Blob Objects

Sometimes the corrupted object is a blob — a specific version of a specific file. If you still have the working tree file, you can reconstruct it:

```
# Figure out which file the broken blob belongs to
# Use the tree object that references it
git ls-tree -r HEAD | grep <broken-sha>
# Output: 100644 blob <broken-sha>    path/to/file.js

# If you have the current version of that file,
# you can hash it back into the object store
git hash-object -w path/to/file.js
```

This only works if the working tree version matches the broken blob. If the file has changed since that commit, you'll need to get the right version from a remote or a backup.

Step 5: The Nuclear Option — Rebuild from Scratch

If the damage is extensive and you have a remote, sometimes the cleanest path is to start fresh:

```
# Rename the broken repo
mv my-project my-project-broken

# Clone fresh from the remote
git clone origin-url my-project

# Copy over any uncommitted work from the broken repo
# Check the working tree for files you hadn't committed yet
diff -rq my-project-broken/ my-project/ --exclude='.git'
```

This isn't really "nuclear" — you're not losing anything that was pushed. The only risk is uncommitted local work, which is why that `diff` at the end matters.

Step 6: Recover Packfile Corruption

This is the nastiest scenario. Git periodically packs loose objects into `.pack` files in `.git/objects/pack/`. If a packfile gets corrupted, you might lose access to hundreds of objects at once.

```
# Try to extract what you can from the damaged pack
git unpack-objects < .git/objects/pack/pack-<hash>.pack
# This will fail on corrupted objects but recover intact ones

# If that doesn't work, try verifying the pack
git verify-pack -v .git/objects/pack/pack-<hash>.idx
# This shows you exactly which objects are in the pack
# and which ones fail verification
```

After extracting what you can, fetch from a remote to fill in the gaps.

Prevention: Avoiding Corruption in the First Place

Once you've lived through a corrupted repo, you get religious about prevention. Here's what I do now:

- **Push frequently.** Every remote is a full backup. I push feature branches even when they're messy — I can always squash later.
- **Don't interrupt Git operations.** If `git gc` or `git repack` is running, let it finish. Killing these mid-operation is the number one cause of corruption I've seen.
- **Avoid shared/networked filesystems for `.git/`.** NFS, CIFS, and cloud-synced folders (Dropbox, OneDrive) are notorious for causing corruption. If you must use them, at least use `git bundle` to create portable backups.
- **Set up `core.fsyncObjectFiles`.** This tells Git to `fsync` objects to disk, protecting against power loss:

```
# Modern Git (2.36+) uses a more granular setting
git config core.fsync objects,derived-metadata,reference
```

- **Run `git fsck` periodically.** Add it to a cron job or a pre-push hook. Catching corruption early gives you more recovery options.

The Bigger Lesson

Git's internal design is genuinely brilliant for recovery. The content-addressable object store means that every clone is a full backup, every object is self-verifying, and corruption is usually localized rather than catastrophic.

The worst Git corruption I ever dealt with took about 20 minutes to fix once I understood the object model. The first time, I spent three hours panicking before I

learned any of this. Hopefully this saves you those three hours.

If you want to go deeper, the [Git Internals chapter](#) in the official Git book is genuinely worth reading. Understanding how `.git/objects/` works turns "my repo is corrupted" from a crisis into a 15-minute fix.



The DEV Team

PROMOTED



Introducing Gemma 4 12B: a unified, encoder-free multimodal model

Bridging the gap between our edge-friendly E4B and our more advanced 26B Mixture of Experts (MoE), Gemma 4 12B packages powerful capabilities inside a reduced memory footprint. It is also our first mid-sized model to feature native audio inputs.

[Read more →](#)

Top comments (5)



Marius-Florin Cristian • Mar 30



In the last 10 months and more actively last 3 days, I went on the journey of making git work inside a FUSE mount, across platforms.

A bit of a pain to make git clone work on a local FUSE mount; a bit of a BIG pain to make git clone, checkout, etc. work in a peered FUSE mount between macos, and linux.

the frigging syscalls, man! i still got some little git lfs issues.



Alan West 🌟 • Mar 30



fuse + git sounds painful. cross platform syscall differences alone would drive me insane. curious what fuse implementation you went with, libfuse or macfuse?



Marius-Florin Cristian • Mar 30 • Edited



for ubuntu went with fuse3, on mac with macfuse, windows, winfsp. went with Bill's (the writer of winfsp) cgofuse for the logic cross platform.

so far, android/ iOS, impossible to use FUSE, as you can't spawn subprocesses with fork(), and as i understood, that is why the fuse api is not exposed.



Marius-Florin Cristian • Mar 30



also I want to point out that git uses mmap a lot while cloning, and performing operations.

this was another FML moment.



Alan West 🌟 • Mar 30



oh god mmap on fuse. yeah that alone would make me question my life choices. the fact that you got it working across three platforms is genuinely impressive though.

[Code of Conduct](#) • [Report abuse](#)

DEV The DEV Team PROMOTED

...



Hacktoberfest applications are open! 🎃

Hacktoberfest is back. 300+ in person Fests and online events worldwide this October, all about building with open-source AI. Host a Fest for your community.

Host a Fest 🎉 →



Alan West

Full-stack developer working with TypeScript, React, and blockchain. Interested in AI-powered dev workflows and open-source tooling.

JOINED

Mar 19, 2026

More from [Alan West](#)

How to build reliable geo-restrictions that actually hold up in production

[#webdev](#) [#security](#) [#backend](#) [#devops](#)

Why your VPS might be part of a botnet — and how to find out

[#security](#) [#devops](#) [#linux](#) [#sysadmin](#)

How to sandbox AI coding agents without crippling them

[#ai](#) [#security](#) [#devops](#) [#docker](#)

DEV The DEV Team **PROMOTED**

...



Hacktoberfest applications are open! 🎃

Hacktoberfest is back. 300+ in person Fests and online events worldwide this October, all about building with open-source AI. Host a Fest for your community.

Host a Fest 🎆 →
